

```
1 void clear2(point *p, int n)
2 {
3     int i, j;
4
5     for (i = 0; i < n; i++) {
6         for (j = 0; j < 3; j++) {
7             p[i].vel[j] = 0;
8             p[i].acc[j] = 0;
9         }
10    }
11 }
```

c) clear2函数

```
1 void clear3(point *p, int n)
2 {
3     int i, j;
4
5     for (j = 0; j < 3; j++) {
6         for (i = 0; i < n; i++) {
7             p[i].vel[j] = 0;
8             for (i = 0; i < n; i++) {
9                 p[i].acc[j] = 0;
10            }
11        }
```

d) clear3函数

图 6-20 (续)

6.3 存储器层次结构

- 6.1 节和 6.2 节描述了存储技术和计算机软件的一些基本的和持久的属性：
- 存储技术：不同存储技术的访问时间差异很大。速度较快的技术每字节的成本要比速度较慢的技术高，而且容量较小。CPU 和主存之间的速度差距在增大。
 - 计算机软件：一个编写良好的程序倾向于展示出良好的局部性。

计算中一个喜人的巧合是，硬件和软件的这些基本属性互相补充得很完美。它们这种相互补充的性质使人想到一种组织存储器系统的方法，称为存储器层次结构 (memory hierarchy)，所有的现代计算机系统中都使用了这种方法。图 6-21 展示了一个典型的存储器层次结构。一般而言，从高层往底层走，存储设备变得更慢、更便宜和更大。在最高层 (L0)，是少量快速的 CPU 寄存器，CPU 可以在一个时钟周期内访问它们。接下来是一个

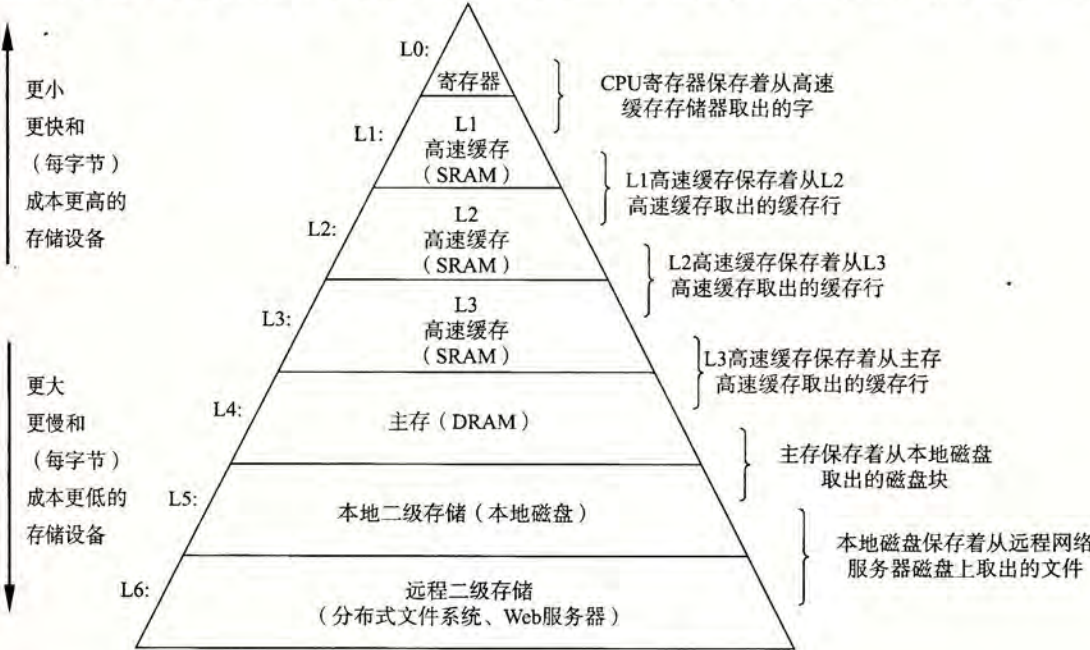


图 6-21 存储器层次结构

或多个小型到中型的基于 SRAM 的高速缓存存储器，可以在几个 CPU 时钟周期内访问它们。然后是一个大的基于 DRAM 的主存，可以在几十到几百个时钟周期内访问它们。接下来是慢速但是容量很大的本地磁盘。最后，有些系统甚至包括了一层附加的远程服务器上的磁盘，要通过网络来访问它们。例如，像安德鲁文件系统 (Andrew File System, AFS) 或者网络文件系统 (Network File System, NFS) 这样的分布式文件系统，允许程序访问存储在远程的网络服务器上的文件。类似地，万维网允许程序访问存储在世界上任何地方的 Web 服务器上的远程文件。

旁注 其他的存储器层次结构

我们向你展示了一个存储器层次结构的示例，但是其他的组合也是可能的，而且确实也很常见。例如，许多站点(包括谷歌的数据中心)将本地磁盘备份到存档的磁带上。其中有些站点，在需要时由人工装好磁带。而其他站点则是由磁带机器人自动地完成这项任务。无论在何种情况中，磁带都是存储器层次结构中的一层，在本地磁盘层下面，本书中提到的通用原则也同样适用于它。磁带每字节比磁盘更便宜，它允许站点将本地磁盘的多个快照存档。代价是磁带的访问时间要比磁盘的更长。来看另一个例子，固态硬盘在存储器层次结构中扮演着越来越重要的角色，连接起 DRAM 和旋转磁盘之间的鸿沟。

6.3.1 存储器层次结构中的缓存

一般而言，高速缓存(cache，读作“cash”)是一个小而快速的存储设备，它作为存储在更大、也更慢的设备中的数据对象的缓冲区域。使用高速缓存的过程称为缓存(caching，读作“cashing”)。

存储器层次结构的中心思想是，对于每个 k ，位于 k 层的更快更小的存储设备作为位于 $k+1$ 层的更大更慢的存储设备的缓存。换句话说，层次结构中的每一层都缓存来自较低一层的数据对象。例如，本地磁盘作为通过网络从远程磁盘取出的文件(例如 Web 页面)的缓存，主存作为本地磁盘上数据的缓存，依此类推，直到最小的缓存——CPU 寄存器组。

图 6-22 展示了存储器层次结构中缓存的一般性概念。第 $k+1$ 层的存储器被划分成连续的数据对象组块(chunk)，称为块(block)。每个块都有一个唯一的地址或名字，使之区别于其他的块。块可以是固定大小的(通常是这样的)，也可以是可变大小的(例如存储在 Web 服务器上的远程 HTML 文件)。例如，图 6-22 中第 $k+1$ 层存储器被划分成 16 个大小固定的块，编号为 0~15。

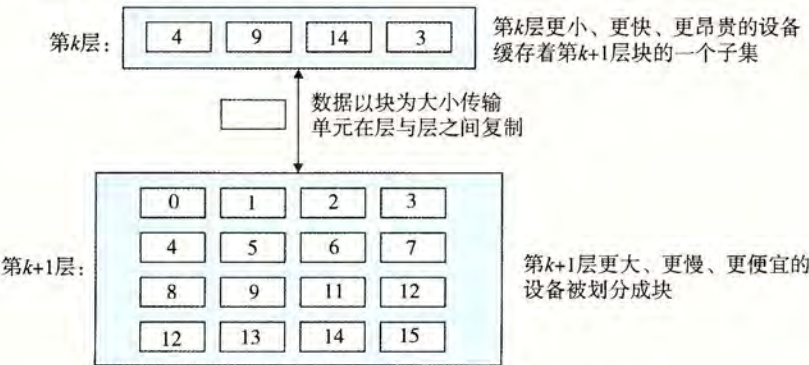


图 6-22 存储器层次结构中基本的缓存原理

类似地,第 k 层的存储器被划分成较少的块的集合,每个块的大小与 $k+1$ 层的块的大小一样。在任何时刻,第 k 层的缓存包含第 $k+1$ 层块的一个子集的副本。例如,在图6-22中,第 k 层的缓存有4个块的空间,当前包含块4、9、14和3的副本。

数据总是以块大小为传送单元(transfer unit)在第 k 层和第 $k+1$ 层之间来回复制的。虽然在层次结构中任何一对相邻的层次之间块大小是固定的,但是其他的层次对之间可以有不同块大小。例如,在图6-21中,L1和L0之间的传送通常使用的是1个字大小的块。L2和L1之间(以及L3和L2之间、L4和L3之间)的传送通常使用的是几十个字节的块。而L5和L4之间的传送用的是大小为几百或几千字节的块。一般而言,层次结构中较低层(离CPU较远)的设备的访问时间较长,因此为了补偿这些较长的访问时间,倾向于使用较大的块。

1. 缓存命中

当程序需要第 $k+1$ 层的某个数据对象 d 时,它首先在当前存储在第 k 层的一个块中查找 d 。如果 d 刚好缓存在第 k 层中,那么就是我们所说的缓存命中(cache hit)。该程序直接从第 k 层读取 d ,根据存储器层次结构的性质,这要比从第 $k+1$ 层读取 d 更快。例如,一个有良好时间局部性的程序可以从块14中读出一个数据对象,得到一个对第 k 层的缓存命中。

2. 缓存不命中

另一方面,如果第 k 层中没有缓存数据对象 d ,那么就是我们所说的缓存不命中(cache miss)。当发生缓存不命中时,第 k 层的缓存从第 $k+1$ 层缓存中取出包含 d 的那个块,如果第 k 层的缓存已经满了,可能就会覆盖现存的一个块。

覆盖一个现存的块的过程称为替换(replacing)或驱逐(evicting)这个块。被驱逐的这个块有时也称为牺牲块(victim block)。决定该替换哪个块是由缓存的替换策略(replacement policy)来控制的。例如,一个具有随机替换策略的缓存会随机选择一个牺牲块。一个具有最近最少被使用(LRU)替换策略的缓存会选择那个最后被访问的时间距现在最远的块。

在第 k 层缓存从第 $k+1$ 层取出那个块之后,程序就能像前面一样从第 k 层读出 d 了。例如,在图6-22中,在第 k 层中读块12中的一个数据对象,会导致一个缓存不命中,因为块12当前不在第 k 层缓存中。一旦把块12从第 $k+1$ 层复制到第 k 层之后,它就会保持在那里,等待稍后的访问。

3. 缓存不命中的种类

区分不同种类的缓存不命中有时候是很有帮助的。如果第 k 层的缓存是空的,那么对任何数据对象的访问都会不命中。一个空的缓存有时被称为冷缓存(cold cache),此类不命中称为强制性不命中(compulsory miss)或冷不命中(cold miss)。冷不命中很重要,因为它们通常是短暂的事件,不会在反复访问存储器使得缓存暖身(warmed up)之后的稳定状态中出现。

只要发生了不命中,第 k 层的缓存就必须执行某个放置策略(placement policy),确定把它从第 $k+1$ 层中取出的块放在哪里。最灵活的替换策略是允许来自第 $k+1$ 层的任何块放在第 k 层的任何块中。对于存储器层次结构中高层的缓存(靠近CPU),它们是用硬件来实现的,而且速度是最优的,这个策略实现起来通常很昂贵,因为随机地放置块,定位起来代价很高。

因此,硬件缓存通常使用的是更严格的放置策略,这个策略将第 $k+1$ 层的某个块限制放置在第 k 层块的一个小的子集中(有时只是一个块)。例如,在图 6-22 中,我们可以确定第 $k+1$ 层的块 i 必须放置在第 k 层的块 $(i \bmod 4)$ 中。例如,第 $k+1$ 层的块 0、4、8 和 12 会映射到第 k 层的块 0;块 1、5、9 和 13 会映射到块 1;依此类推。注意,图 6-22 中的示例缓存使用的就是这个策略。

这种限制性的放置策略会引起一种不命中,称为冲突不命中(conflict miss),在这种情况下,缓存足够大,能够保存被引用的数据对象,但是因为这些对象会映射到同一个缓存块,缓存会一直不命中。例如,在图 6-22 中,如果程序请求块 0,然后块 8,然后块 0,然后块 8,依此类推,在第 k 层的缓存中,对这两个块的每次引用都会不命中,即使这个缓存总共可以容纳 4 个块。

程序通常是按照一系列阶段(如循环)来运行的,每个阶段访问缓存块的某个相对稳定不变的集合。例如,一个嵌套的循环可能会反复地访问同一个数组的元素。这个块的集合称为这个阶段的工作集(working set)。当工作集的大小超过缓存的大小时,缓存会经历容量不命中(capacity miss)。换句话说就是,缓存太小了,不能处理这个工作集。

4. 缓存管理

正如我们提到过的,存储器层次结构的本质是,每一层存储设备都是较低一层的缓存。在每一层上,某种形式的逻辑必须管理缓存。这里,我们的意思是指某个东西要将缓存划分成块,在不同的层之间传送块,判定是命中还是不命中,并处理它们。管理缓存的逻辑可以是硬件、软件,或是两者的结合。

例如,编译器管理寄存器文件,缓存层次结构的最高层。它决定当发生不命中时何时发射加载,以及确定哪个寄存器来存放数据。L1、L2 和 L3 层的缓存完全是由内置在缓存中的硬件逻辑来管理的。在一个有虚拟内存的系统中,DRAM 主存作为存储在磁盘上的数据块的缓存,是由操作系统软件和 CPU 上的地址翻译硬件共同管理的。对于一个具有像 AFS 这样的分布式文件系统的机器来说,本地磁盘作为缓存,它是由运行在本地机器上的 AFS 客户端进程管理的。在大多数时候,缓存都是自动运行的,不需要程序采取特殊的或显式的行动。

6.3.2 存储器层次结构概念小结

概括来说,基于缓存的存储器层次结构行之有效,是因为较慢的存储设备比较快的存储设备更便宜,还因为程序倾向于展示局部性:

- 利用时间局部性:由于时间局部性,同一数据对象可能会被多次使用。一旦一个数据对象在第一次不命中时被复制到缓存中,我们就会期望后面对该目标有一系列的访问命中。因为缓存比低一层的存储设备更快,对后面的命中的服务会比最开始的不命中快很多。
- 利用空间局部性:块通常包含有多个数据对象。由于空间局部性,我们会期望后面对该块中其他对象的访问能够补偿不命中后复制该块的花费。

现代系统中到处都使用了缓存。正如从图 6-23 中能够看到的那样,CPU 芯片、操作系统、分布式文件系统中 and 万维网上都使用了缓存。各种各样硬件和软件的组合构成和管理着缓存。注意,图 6-23 中有大量我们还未涉及的术语和缩写。在此我们包括这些术语和缩写是为了说明缓存是多么的普遍。

类型	缓存什么	被缓存在何处	延迟（周期数）	由谁管理
CPU寄存器	4字节或8字节字	芯片上的CPU寄存器	0	编译器
TLB	地址翻译	芯片上的TLB	0	硬件MMU
L1高速缓存	64字节块	芯片上的L1高速缓存	4	硬件
L2高速缓存	64字节块	芯片上的L2高速缓存	10	硬件
L3高速缓存	64字节块	芯片上的L3高速缓存	50	硬件
虚拟内存	4KB页	主存	200	硬件 + OS
缓冲区缓存	部分文件	主存	200	OS
磁盘缓存	磁盘扇区	磁盘控制器	100 000	控制器固件
网络缓存	部分文件	本地磁盘	10 000 000	NFS客户
浏览器缓存	Web页	本地磁盘	10 000 000	Web浏览器
Web缓存	Web页	远程服务器磁盘	1 000 000 000	Web代理服务器

图 6-23 缓存在现代计算机系统中无处不在。TLB：翻译后备缓冲器(Translation Lookaside Buffer)；MMU：内存管理单元(Memory Management Unit)；OS：操作系统(Operating System)；AFS：安德鲁文件系统(Andrew File System)；NFS：网络文件系统(Network File System)

6.4 高速缓存存储器

早期计算机系统的存储器层次结构只有三层：CPU 寄存器、DRAM 主存储器和磁盘存储。不过，由于 CPU 和主存之间逐渐增大的差距，系统设计者被迫在 CPU 寄存器文件和主存之间插入了一个小的 SRAM 高速缓存存储器，称为 L1 高速缓存(一级缓存)，如图 6-24 所示。L1 高速缓存的访问速度几乎和寄存器一样快，典型地是大约 4 个时钟周期。

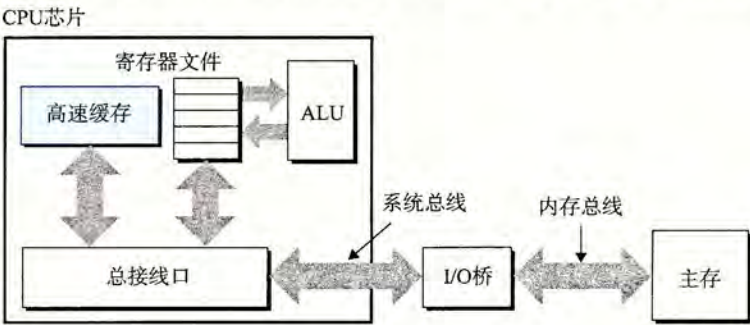


图 6-24 高速缓存存储器的典型总线结构

随着 CPU 和主存之间的性能差距不断增大，系统设计者在 L1 高速缓存和主存之间又插入了一个更大的高速缓存，称为 L2 高速缓存，可以在大约 10 个时钟周期内访问到它。有些现代系统还包括有一个更大的高速缓存，称为 L3 高速缓存，在存储器层次结构中，它位于 L2 高速缓存和主存之间，可以在大约 50 个周期内访问到它。虽然安排上有相当多的变化，但是通用原则是一样的。对于下一节中的讨论，我们会假设一个简单的存储器层次结构，CPU 和主存之间只有一个 L1 高速缓存。

6.4.1 通用的高速缓存存储器组织结构

考虑一个计算机系统，其中每个存储器地址有 m 位，形成 $M=2^m$ 个不同的地址。如图 6-25a 所示，这样一个机器的高速缓存被组织成一个有 $S=2^s$ 个高速缓存组(cache set)的